

Learning the Denoiser by Expectation Maximization

Belief propagation as an exact E-step for Markov diffusion priors

Thesis project — Layer 5 working note

July 2026

Abstract

Earlier layers of this project established that when the clean data is a Markov chain, belief propagation computes the diffusion score *exactly*: the posterior induced by coordinatewise noising is again a chain, chains are trees, and sum-product BP on a tree is exact. That result assumed the prior was known. This note removes the assumption. We treat the transition kernel of the clean chain as an unknown parameter, observe only noised sequences, and estimate the kernel by maximum marginal likelihood via expectation maximization.

Four things make this more than a routine application of EM. First, the E-step is *exact*, not variational: the same tree structure that made the score exact makes the posterior expectations exact. Second, the entire E-step over an arbitrarily large dataset compresses into a single $M \times M$ matrix Ξ , the continuous analogue of the expected transition-count matrix of Baum–Welch, which is a sufficient statistic independent of how the kernel is parameterized. Third, Fisher’s identity turns Ξ into the exact gradient of the marginal log-likelihood, so *nothing is ever differentiated through the forward–backward recursion* — BP supplies the expectation rather than being backpropagated through. Fourth, and this is the point of the exercise, the learned object lives on $\mathbb{R} \times \mathbb{R}$ and carries no dependence on the diffusion time: one fit yields the exact Bayes denoiser at *every* noise level, whereas a score network must learn a function on $\mathbb{R}^n \times \mathbb{R}_+$ and be trained across the whole schedule.

We also make precise a remark that motivated the exercise — that learning the prior “does not require noising at all”. Noising never destroys identifiability, because the Gaussian characteristic function has no zeros (Proposition 4); what it destroys is *information*, at a rate we compute and measure. Section 5 states the efficiency argument as a risk decomposition and Section 6 collects the limitations, of which the honest one is that BP is dramatically slower than a network at inference time.

Contents

1	Setting	2
1.1	The two processes	2
1.2	The prior, now with unknown parameters	3
2	Exact EM through belief propagation	3
2.1	The EM decomposition	3
2.2	The E-step is exact	4
2.3	The whole E-step is one matrix	4
2.4	Gradients without differentiating through BP	5
2.5	The algorithm and its cost	6

3	What noising costs, and what it does not	6
3.1	The two limits	6
3.2	Identifiability survives any finite noise level	7
3.3	The information budget	7
4	Parameterizations and their M-steps	7
4.1	Rung 1 — Gaussian AR(1): the correctness harness	8
4.2	Rung 2 — Laplace AR(1): closed form despite a non-smooth kernel	8
4.3	Rung 3 — unknown innovation law	9
4.4	Rung 4 — a neural transition kernel	9
5	Why this should beat a score network	11
5.1	The decoupling	11
5.2	Risk decomposition	11
5.3	What the comparison must control	12
6	Limitations	12
7	Relation to the rest of the project	13
A	Notation	14
B	Implementation map	14

1 Setting

1.1 The two processes

Throughout, the clean datum is a real sequence $a = (a_1, \dots, a_n) \in \mathbb{R}^n$ drawn from a prior p_0 , and the forward (noising) process is the coordinatewise variance-preserving Ornstein–Uhlenbeck SDE

$$dX_s = -X_s ds + \sqrt{2} dW_s, \quad X_0 = a, \quad (1)$$

whose transition kernel at diffusion time t is

$$X_t \mid a \sim \mathcal{N}(\alpha_t a, \Delta_t I), \quad \alpha_t = e^{-t}, \quad \Delta_t = 1 - e^{-2t}. \quad (2)$$

For unit-variance clean data $\alpha_t^2 + \Delta_t = 1$, so the noisy marginal has unit variance at every t . We write x for a realization of X_t and p_t for its density.

Two facts from the earlier layers are used without reproof. The first is the *score identity*: because the channel (2) is linear-Gaussian,

$$s(x, t) = \nabla_x \log p_t(x) = -\frac{x - \alpha_t m(x, t)}{\Delta_t}, \quad m(x, t) = \mathbb{E}[a \mid X_t = x], \quad (3)$$

so estimating the score and estimating the posterior mean (the *denoiser*) are the same problem, and an error in one transfers to the other by the exact relation $\hat{s} - s = (\alpha_t/\Delta_t)(\hat{m} - m)$. The second is the *exactness result*: if p_0 is Markov on the chain $1 - 2 - \dots - n$, then so is the posterior $p_t(a \mid x)$, and sum–product BP returns its marginals exactly.

1.2 The prior, now with unknown parameters

We take the clean prior to be a chain with a parameterized transition kernel,

$$p_\theta(a) = \mu(a_1) \prod_{i=2}^n K_\theta(a_i | a_{i-1}), \quad \theta \in \Theta \subseteq \mathbb{R}^P, \quad (4)$$

with μ known. (Learning μ requires no new machinery — it uses the single-site belief at $i = 1$ in place of the pairwise beliefs — and is omitted only to keep the notation light. In the experiments $\mu = \mathcal{N}(0, 1)$ is genuinely the law of a_1 under the data-generating process, so this is not an approximation.)

The data available for learning is a sample of *noisy* sequences,

$$\mathcal{D} = \{ (x^{(d)}, t_d) \}_{d=1}^N, \quad x^{(d)} \sim p_{t_d, \theta^*}, \quad a^{(d)} \sim p_{\theta^*} \text{ never observed.} \quad (5)$$

Note what is and is not known: the observation channel (2) is known exactly, including t_d , and the *structure* of the prior (a chain) is known; the *content* of the prior, K_θ , is not. This is the situation the project cares about — “the data is Markov plus noise, but we do not know the model behind it” — and it is strictly harder than the setting of the earlier layers, where K was handed to BP.

The estimator is maximum marginal likelihood,

$$\hat{\theta} = \arg \max_{\theta} L(\theta), \quad L(\theta) = \sum_{d=1}^N \log p_{t_d, \theta}(x^{(d)}), \quad p_{t, \theta}(x) = \int_{\mathbb{R}^n} p_\theta(a) \prod_{i=1}^n \ell_i^t(a_i; x_i) da, \quad (6)$$

where the per-site likelihood factor implied by (2) is

$$\ell_i^t(a_i; x_i) = \mathcal{N}(x_i; \alpha_t a_i, \Delta_t). \quad (7)$$

The integral in (6) is over the unobserved clean chain: this is a latent-variable problem, and EM is the natural way to attack it.

Remark 1 (Why this is the right kind of “learning”). The hypothesis space here is not a function class but the set of Markov chains. The estimator cannot represent a non-Markov prior at all, and it cannot represent a denoiser that fails to be the exact Bayes denoiser of *some* chain. That is a severe restriction, and it is exactly the point: the restriction is known to be satisfied (or nearly so) by the data, so imposing it converts a nonparametric function-estimation problem into a low-dimensional parametric one.

2 Exact EM through belief propagation

2.1 The EM decomposition

Fix a current parameter θ' and write $p_{\theta'}(a | x)$ for the posterior over the clean chain. The standard decomposition of the marginal log-likelihood reads, for a single observation x at level t ,

$$\log p_{t, \theta}(x) = \underbrace{\mathbb{E}_{\theta'}[\log p_\theta(a, x)]}_{Q(\theta | \theta')} + \underbrace{H(p_{\theta'}(\cdot | x))}_{\text{free of } \theta} + \text{KL}(p_{\theta'}(\cdot | x) \| p_\theta(\cdot | x)), \quad (8)$$

where $\mathbb{E}_{\theta'}$ is taken under $p_{\theta'}(a | x)$ and $p_\theta(a, x) = p_\theta(a) \prod_i \ell_i^t(a_i; x_i)$. Since the likelihood factors (7) do not involve θ , they contribute a constant, and summing over the dataset gives

$$Q(\theta | \theta') = \sum_{d=1}^N \sum_{i=2}^n \mathbb{E}_{\theta'} [\log K_\theta(a_i^{(d)} | a_{i-1}^{(d)})] + \text{const.} \quad (9)$$

Two things follow immediately, and they organize everything below. First, Q depends on the posterior only through the *pairwise* marginals of adjacent sites. Second, because the KL term in (8) is non-negative and vanishes at $\theta = \theta'$,

$$Q(\theta \mid \theta') \geq Q(\theta' \mid \theta') \implies L(\theta) \geq L(\theta'), \quad (10)$$

which is the monotone-ascent guarantee. We use (10) as the project's correctness test: an error anywhere in the forward-backward recursion, the pairwise accumulation, or the M-step generically breaks it.

2.2 The E-step is exact

Proposition 1 (Exact E-step). *Let p_θ be the chain prior (4) and let the observation likelihood factorize sitewise as $\prod_i \ell_i(a_i)$. Then the posterior*

$$p_\theta(a \mid x) \propto \mu(a_1) \ell_1(a_1) \prod_{i=2}^n K_\theta(a_i \mid a_{i-1}) \ell_i(a_i) \quad (11)$$

is a Markov chain on the same graph, and sum-product BP computes its single-site and pairwise marginals exactly.

Proof. The likelihood factors are unary, so they reweight node potentials without introducing edges: the factor graph of the posterior is the factor graph of the prior with modified node potentials. That graph is a chain, hence a tree, and sum-product BP is exact on trees. Exactness of the pairwise marginals on tree edges is the standard consequence for the edge beliefs. $\square$

This is the structural payoff of the whole project. In a generic latent-variable model the E-step is intractable and must be approximated — by loopy BP, by expectation propagation, by a variational bound — and the monotonicity guarantee (10) is lost along with the exactness. Here it is not approximated at all. The only error is the numerical representation of the continuous messages, which Layer 2 pinned at the 10^{-15} floor for the grids used.

Concretely, writing L_i and R_i for the forward and backward messages arriving at site i (each *excluding* the local likelihood), the recursions and beliefs are

$$L_{i+1}(a') = \int K_\theta(a' \mid a) L_i(a) \ell_i(a) da, \quad R_{i-1}(a) = \int K_\theta(a' \mid a) R_i(a') \ell_i(a') da', \quad (12)$$

$$b_i(a) \propto L_i(a) \ell_i(a) R_i(a), \quad b_{i-1,i}(a, a') \propto L_{i-1}(a) \ell_{i-1}(a) K_\theta(a' \mid a) \ell_i(a') R_i(a'). \quad (13)$$

2.3 The whole E-step is one matrix

Represent the messages on a grid $u_1, \dots, u_M$ with quadrature weights $w_1, \dots, w_M$, and define for each chain d and each site i the vectors

$$f_i^{(d)}(j) = w_j L_i^{(d)}(u_j) \ell_i^{(d)}(u_j), \quad g_i^{(d)}(k) = w_k R_i^{(d)}(u_k) \ell_i^{(d)}(u_k), \quad (14)$$

i.e. the incoming messages *with* their local factors folded in. Writing $K[k, j] = K_\theta(u_k \mid u_j)$, the pairwise belief (13) on edge $(i, i+1)$ evaluated at the grid pair (u_j, u_k) is $f_i^{(d)}(j) K[k, j] g_{i+1}^{(d)}(k) / Z_i^{(d)}$ with $Z_i^{(d)} = (g_{i+1}^{(d)})^\top K f_i^{(d)}$.

Proposition 2 (Sufficiency of the pair-count matrix). *Define*

$$C = \sum_{d=1}^N \sum_{i=1}^{n-1} \frac{g_{i+1}^{(d)} (f_i^{(d)})^\top}{Z_i^{(d)}} \in \mathbb{R}^{M \times M}, \quad \Xi = C \odot K, \quad (15)$$

with $\odot$ the entrywise product. Then $\Xi[k, j]$ is the total posterior mass the dataset places on the transition $u_j \mapsto u_k$; it satisfies $\sum_{j,k} \Xi[k, j] = N(n-1)$; and the expected complete-data log-likelihood (9) is the bilinear form

$$Q(\theta \mid \theta') = \langle \Xi(\theta'), \log K_\theta \rangle + \text{const}, \quad \langle A, B \rangle = \sum_{j,k} A[k, j] B[k, j]. \quad (16)$$

Proof. Immediate from (9) and the grid form of (13): each edge contributes its normalized pairwise belief, which sums to one over the grid, and Q is the sum over edges of $\sum_{j,k} b_{i,i+1}(u_j, u_k) \log K_\theta[k, j]$. Collecting the belief mass across edges and chains gives Ξ . $\square$

Proposition 2 is the algorithmic heart of the method, and it is worth stating what it buys.

1. **Ξ is a sufficient statistic of the E-step.** It does not depend on how K_θ is parameterized. A scalar AR(1), a mixture, a neural network — all consume the same $M \times M$ matrix. Changing the parameterization changes only the M-step.
2. **The M-step never touches the data.** After the E-step, the dataset can be discarded. The M-step cost is $O(PM^2)$ regardless of N .
3. **Different noise levels simply add.** The level t enters only through ℓ_i^t ; statistics from observations at different t accumulate into one Ξ . A dataset spread over the whole diffusion schedule is summarized by a single matrix.
4. **It is the continuum Baum–Welch statistic.** For a discrete-state HMM, Ξ is exactly the matrix of expected transition counts. The continuous case differs only in that the state space must be represented.

2.4 Gradients without differentiating through BP

The M-step maximizes (16). When no closed form is available one follows the gradient, and here the relevant gradient is available exactly and cheaply.

Proposition 3 (Fisher’s identity for the chain). *For any θ at which $\log K_\theta$ is differentiable,*

$$\nabla_\theta L(\theta) = \langle \Xi(\theta), \nabla_\theta \log K_\theta \rangle, \quad (17)$$

where $\Xi(\theta)$ is the pair-count matrix (15) assembled at the same θ . In particular the gradient of the exact marginal log-likelihood requires one forward–backward pass and no differentiation of the BP recursion.

Proof. For one observation, $\nabla_\theta \log p_\theta(x) = p_\theta(x)^{-1} \int \nabla_\theta p_\theta(a, x) da = \int p_\theta(a \mid x) \nabla_\theta \log p_\theta(a, x) da = \mathbb{E}_\theta[\nabla_\theta \log p_\theta(a)]$, the likelihood factors being θ -free. Since $\nabla_\theta \log p_\theta(a) = \sum_i \nabla_\theta \log K_\theta(a_i \mid a_{i-1})$ involves only adjacent pairs, the expectation is taken under the pairwise posteriors, and summing over the dataset gives (17). Equivalently: $\nabla_\theta L(\theta) = \nabla_\theta Q(\theta \mid \theta')|_{\theta'=\theta}$. $\square$

Remark 2 (This is the computational point). It is tempting to implement “learning inside BP” by writing the forward–backward recursion in an autodiff framework and backpropagating the evidence through it. Proposition 3 says this is unnecessary and wasteful: the derivative of the log-evidence with respect to the parameters is an *expectation under the beliefs BP already computed*. BP is not a computation graph to differentiate; it is the oracle that supplies the expectation. The implementation accompanying this note is plain `numpy`, with no autodiff anywhere, and its gradients agree with finite differences of the exact evidence to $\sim 10^{-9}$ relative for smooth kernels (Section 4 records the one exception).

2.5 The algorithm and its cost

Algorithm 1 EM for a chain prior observed through the OU channel

```

1: input noisy chains grouped by level  $\{(X^{(g)}, \alpha_g, \Delta_g)\}$ , grid  $u$ , weights  $w$ , initial  $\theta^{(0)}$ 
2: for  $r = 0, 1, 2, \dots$  do
3:    $K \leftarrow \exp \log K_{\theta^{(r)}}(u)$   $\triangleright O(PM^2)$ , once per iteration
4:    $\Xi \leftarrow 0, L \leftarrow 0$ 
5:   for each group  $g$  and each chain  $d$  in it do
6:     forward–backward pass (12)  $\triangleright O(nM^2)$  per chain, batched as matmuls
7:     accumulate  $\Xi$  by (15) and the exact  $\log p_{t_g, \theta^{(r)}}(x^{(d)})$  into  $L$ 
8:   end for
9:    $\theta^{(r+1)} \leftarrow \arg \max_{\theta} \langle \Xi, \log K_{\theta} \rangle$   $\triangleright$  closed form, or an ascent step (generalized EM)
10:  assert  $L$  non-decreasing  $\triangleright$  the correctness test (10)
11: end for

```

Per iteration the cost is $O(NnM^2)$ for the E-step and $O(PM^2)$ for the M-step. The M^2 is the price of representing continuous messages; the important scalings are that the cost is *linear* in the dataset size and *independent of the parameter count* in the dominant term. A score network inverts both: its cost per gradient step is proportional to the parameter count and it needs many passes over the data.

3 What noising costs, and what it does not

The framing that motivated this note included the remark that learning the prior parameters “does not require noising at all”. That is correct, and it is worth saying precisely why, because the precise version is more useful than the slogan.

3.1 The two limits

At $t \rightarrow 0$ we have $\alpha_t \rightarrow 1$, $\Delta_t \rightarrow 0$, and the likelihood factor (7) tends to $\delta(a_i - x_i)$. The posterior collapses onto the observation, the pairwise beliefs become products of deltas, Ξ degenerates into a histogram of *observed* transitions, and (16) becomes the ordinary complete-data log-likelihood. EM degenerates to plain maximum likelihood on clean data — no latent variables, no iteration required for kernels with closed-form MLEs. At the other end, $t \rightarrow \infty$ gives $\alpha_t \rightarrow 0$: the observation is independent of a , the posterior equals the prior, and $\nabla_{\theta} L \rightarrow 0$. Nothing can be learned.

So noising is not needed to identify the prior; the clean chain, if available, identifies it more efficiently. Noising is what the *problem* imposes when only noisy data exists — and, more interestingly, it is a knob we can turn.

3.2 Identifiability survives any finite noise level

Proposition 4 (Noising preserves identifiability). *Fix $t \in (0, \infty)$ and let P_θ denote the law of the clean sequence $a \in \mathbb{R}^n$ under (4). If $p_{t,\theta} = p_{t,\theta'}$ as distributions on $\mathbb{R}^n$, then $P_\theta = P_{\theta'}$. Consequently the model is identifiable at noise level t if and only if it is identifiable from clean data.*

Proof. Under (2), $X_t = \alpha_t a + \sqrt{\Delta_t} z$ with $z \sim \mathcal{N}(0, I)$ independent of a , so the characteristic functions satisfy

$$\varphi_{t,\theta}(\xi) = \varphi_\theta(\alpha_t \xi) e^{-\Delta_t \|\xi\|^2/2}, \quad \xi \in \mathbb{R}^n, \quad (18)$$

where φ_θ is the characteristic function of P_θ . The Gaussian factor $e^{-\Delta_t \|\xi\|^2/2}$ is strictly positive for every ξ , and $\alpha_t = e^{-t} > 0$ for every finite t . Hence $\varphi_{t,\theta} = \varphi_{t,\theta'}$ forces $\varphi_\theta(\alpha_t \xi) = \varphi_{\theta'}(\alpha_t \xi)$ for all ξ , i.e. $\varphi_\theta = \varphi_{\theta'}$ on $\mathbb{R}^n$, and characteristic functions determine distributions. $\square$

Remark 3 (Injective but ill-conditioned). Proposition 4 is a statement about population distributions, and it is exactly as strong — and as weak — as the classical fact that Gaussian deconvolution is injective. Injectivity does not mean stability: the inverse map is unbounded, and dividing by $e^{-\Delta_t \|\xi\|^2/2}$ amplifies high frequencies catastrophically. Finite-sample behaviour is therefore governed not by identifiability but by *information*, which is the subject of the next subsection. The practical reading is: no noise level is fatal in principle, but the estimator’s variance grows quickly with t , and features of P_θ living at high frequency — sharp peaks, heavy tails, the very non-Gaussianity this project is about — are the first to be lost.

3.3 The information budget

The relevant finite-sample quantity is the Fisher information of θ carried by one noisy chain,

$$J_t = \mathbb{E}_{x \sim p_{t,\theta}} \left[\nabla_\theta \log p_{t,\theta}(x) \nabla_\theta \log p_{t,\theta}(x)^\top \right], \quad (19)$$

which by Proposition 3 is computable exactly from single-chain BP passes: the per-chain score is $\langle \Xi(x), \nabla_\theta \log K_\theta \rangle$, requiring no simulation of the estimator and no numerical differentiation. The Cramér–Rao bound then gives $\text{Var}(\hat{\theta}_p) \gtrsim [J_t^{-1}]_{pp}/N$ for unbiased estimators, which we use as an order-of-magnitude yardstick rather than a strict bound (the maximum-likelihood estimator is biased at finite N , and the measured bias is reported alongside the variance in the experiments).

J_t is monotonically destroyed as t grows. The measured decay, together with the realized estimator error, is the content of Part 2 of experiment 06: the point of that comparison is to check the estimator against *its own information budget*, so that a large error at large t is correctly read as “the data does not contain this” rather than “the algorithm is failing”.

Corollary 1 (The remark, made precise). *If clean sequences are available, estimate θ from them directly: this is the $t \rightarrow 0$ limit, it is the most informative case, and it requires no BP at all during fitting. The exact denoiser at every noise level then follows by running BP with $\hat{\theta}$, with no further learning. If only noisy sequences are available, the same estimator is still consistent for any finite t (Proposition 4) and EM with an exact BP E-step computes it.*

Both regimes are implemented and compared: `fit_clean` builds Ξ as a histogram of observed transitions, `fit_em` builds it from beliefs, and the downstream M-step code is identical.

4 Parameterizations and their M-steps

Proposition 2 isolates all the modelling choice into the M-step. We implement four rungs of expressivity, in increasing order.

4.1 Rung 1 — Gaussian AR(1): the correctness harness

$K_\theta(a' | a) = \mathcal{N}(a'; \rho a, q)$, $\theta = (\rho, q)$. Setting $\nabla_\theta Q = 0$ in (16) gives, with $S_{ab} = \sum_{j,k} \Xi[k, j] u_j^a u_k^b$ and $W = \sum_{j,k} \Xi[k, j]$,

$$\rho^+ = \frac{S_{11}}{S_{20}}, \quad q^+ = \frac{S_{02} - (S_{11})^2/S_{20}}{W}, \quad (20)$$

the belief-weighted Yule–Walker equations. In the $t \rightarrow 0$ limit Ξ becomes a transition histogram and (20) reduces to ordinary AR(1) least squares, as it must. This rung exists to be checked against closed forms: the whole prior is Gaussian, so the evidence, the score and the posterior mean are all available analytically.

4.2 Rung 2 — Laplace AR(1): closed form despite a non-smooth kernel

$K_\theta(a' | a) = \frac{1}{2b} \exp(-|a' - \rho a|/b)$, $\theta = (\rho, b)$. Here

$$Q(\rho, b) = -\frac{1}{b} \sum_{j,k} \Xi[k, j] |u_k - \rho u_j| - W \log 2b. \quad (21)$$

For fixed ρ the optimal scale is the belief-weighted mean absolute residual, $b(\rho) = W^{-1} \sum \Xi[k, j] |u_k - \rho u_j|$; substituting back gives $Q(\rho) = -W \log 2b(\rho) - W$, so maximizing over ρ is *minimizing a weighted mean absolute residual* — a least-absolute-deviations regression through the origin. Its exact solution is the weighted median of the ratios u_k/u_j with weights $\Xi[k, j] |u_j|$. The joint M-step is therefore closed form even though the kernel is not differentiable: no line search, no gradient.

Remark 4 (A discretization caveat, and why it is not generic). Two things go wrong for this kernel on a uniform grid through the origin, and both are artifacts of the discretization rather than of EM.

(i) *Gradient accuracy.* $\partial_\rho \log K_\theta$ contains $\text{sign}(a' - \rho a)$, a discontinuity across the diagonal. Trapezoidal quadrature of a discontinuous integrand loses the spectral accuracy the rest of this package enjoys: against a finite difference of the exact grid evidence, the ρ -component of (17) is off by $\sim 13\%$ at $M = 201$ and still $\sim 0.1\%$ at $M = 1601$, decaying erratically. The b -component, which is smooth, agrees to machine precision — confirming that Proposition 3 is fine and the quadrature is not.

(ii) *Lattice snapping.* The minimizer of a weighted sum of $|u_k - \rho u_j|$ is a breakpoint, i.e. one of the ratios u_k/u_j . On a uniform grid through the origin these ratios are rationals m/l , and a low-denominator value such as $4/5$ has many aliases ($8/10, 12/15, \dots$) that pool weight onto it. So $\hat{\rho}$ lands on simple rationals: spuriously *exactly* correct when the truth happens to be one, and stuck at lattice resolution when it is not. Experiment 06 Part 5 measures this against M and against ρ^* ; the reported $N^{-1/2}$ rates use the smooth kernels, whose M-steps are ratios of smooth moments and vary continuously.

Remark 5 (The literal gradient route, measured). The suggestion that started this layer was phrased as gradient ascent — iterate along $\nabla_\theta L$ from a random start — rather than as EM. Fisher’s identity makes that route immediately available, since (17) is the gradient of the exact marginal likelihood from the same single BP pass. Both routes share the E-step and the statistic Ξ ; they differ only in what they do with it:

$$\text{gradient: } \theta \leftarrow \theta + \eta \langle \Xi, \nabla_\theta \log K_\theta \rangle, \quad \text{EM: } \theta \leftarrow \arg \max_{\theta} \langle \Xi, \log K_\theta \rangle. \quad (22)$$

Experiment 08 measures them side by side (256 chains, $n = 24$, 120 iterations), and the answer is not one-sided.

On the smooth Gaussian kernel the gradient route *converges to the same place*: at $\eta = 0.5$ it matches EM’s final log-likelihood to 2×10^{-9} nats and its parameter errors to four digits. At $\eta = 0.1$ it is simply not there yet (1.6 nats short); at $\eta = 2$ it diverges, violating monotonicity by 527 nats; at $\eta = 8$ it leaves the admissible set entirely. Nothing is gained over EM here, and a step size must be tuned to lose nothing.

On the Laplace kernel the gradient route is genuinely *better*. It reaches a higher log-likelihood than the exact M-step (-8064.7 against -8066.6 at $\eta = 0.1$) and better parameters on both coordinates ($|\Delta\rho| = 0.022$ against 0.050 , $|\Delta b| = 0.013$ against 0.027). The reason is Remark 4: EM’s M-step here is exactly optimal for the discretized model and lands on a lattice point, reaching a genuine fixed point after 72 iterations with $\hat{\rho}$ stuck 0.05 from the truth, while gradient steps move continuously and walk past it. Divergence, when it happens, is unmistakable rather than subtle — at $\eta = 8$ the scale collapses toward a near-delta kernel and the reported log-density goes positive.

So the two are complementary rather than ranked, and the split is not the one intuition suggests. The exact M-step is step-size-free and exactly monotone, and should be preferred wherever it exists *and* the kernel is smooth. The gradient route earns its place precisely where the M-step is least trustworthy: where it has no closed form (Rung 4), or where its exact solution is an artifact of the discretization rather than of the model (Rung 2). What is common to both, and is the point of Proposition 3, is that neither ever differentiates through the forward–backward recursion.

4.3 Rung 3 — unknown innovation law

Keep the linear autoregression but make the innovation density essentially free:

$$K_\theta(a' | a) = \sum_{c=1}^C \pi_c \mathcal{N}(a' - \rho a; \mu_c, \sigma_c^2), \quad \theta = (\rho, \{\pi_c, \mu_c, \sigma_c^2\}_{c=1}^C). \quad (23)$$

This is the estimator to use when the model behind the data is genuinely unknown — it has never heard of the Laplace density it will be asked to recover. The complete-data problem now carries a *second* latent variable, the mixture label, so the M-step is itself an EM, run against the fixed Ξ . With responsibilities $r_c(j, k)$ computed at the current parameters and $e_{jk} = u_k - \rho u_j$, each block is closed form:

$$\pi_c^+ = \frac{\langle \Xi, r_c \rangle}{W}, \quad \mu_c^+ = \frac{\langle \Xi, r_c e \rangle}{\langle \Xi, r_c \rangle}, \quad (\sigma_c^2)^+ = \frac{\langle \Xi, r_c (e - \mu_c^+)^2 \rangle}{\langle \Xi, r_c \rangle}, \quad \rho^+ = \frac{\sum_c \sigma_c^{-2} \langle \Xi, r_c u_j (u_k - \mu_c) \rangle}{\sum_c \sigma_c^{-2} \langle \Xi, r_c u_j^2 \rangle}. \quad (24)$$

Alternating these blocks is expectation conditional maximization, which is monotone; since Ξ is fixed, each inner sweep is $O(CM^2)$ on a matrix that does not grow with the dataset.

Remark 6 (A constraint that looks free and is not). The innovation of a zero-mean stationary chain should satisfy $\sum_c \pi_c \mu_c = 0$. Imposing it by recentering the component means after each update is *not* harmless: the projection does not maximize Q , and in testing it broke monotone ascent by $\sim 10^{-2}$ nats. Left unconstrained, ECM is exactly monotone and the fitted innovation mean comes out at the 10^{-3} level by itself. We therefore report the innovation mean as a diagnostic rather than enforcing it — a small illustration of the general rule that every step of an EM must be justified as an ascent step, not merely as a reasonable-looking adjustment.

4.4 Rung 4 — a neural transition kernel

The final rung answers the “small deep network” suggestion directly, but places the network *inside* the exact recursion rather than in place of it. Let a small MLP ϕ map a state to the parameters of

a conditional mixture,

$$K_\phi(a' | a) = \sum_{c=1}^C \pi_c(a) \mathcal{N}(a'; m_c(a), \sigma_c^2(a)), \quad (\pi, m, \log \sigma^2)(a) = \text{MLP}_\phi(a), \quad (25)$$

a mixture-density network, nonparametric in both the location and the shape of the transition. There is no closed-form M-step, so we take one Adam step on Q per E-step — generalized EM, which only requires Q to increase, and which therefore requires *checking* that it does; see Remark 7. The gradient is where the structure pays off. By the chain rule applied to (16),

$$\frac{\partial Q}{\partial \phi} = \sum_j \left[\sum_k \tilde{\Xi}[k, j] \frac{\partial \log K_\phi[k, j]}{\partial \text{head}(u_j)} \right] \frac{\partial \text{head}(u_j)}{\partial \phi}, \quad (26)$$

so **the network is evaluated and backpropagated only at the M grid points**, once per EM iteration, *no matter how large the dataset is*. All dependence on the data has been absorbed into Ξ . A vanilla score network, by contrast, must backpropagate through every training example at every step. Here $\tilde{\Xi}[k, j] = \Xi[k, j] - (\sum_k \Xi[k, j])p_j(k)$ is the centered statistic, p_j being the grid-normalized kernel column; the subtracted term is the derivative of the column log-partition, the familiar “observed minus expected” form. Verified against finite differences of the exact evidence to 1.6×10^{-6} relative.

Remark 7 (A fixed learning rate is not an ascent step). “Generalized EM only needs Q to increase” is a licence to take a cheap M-step, not a licence to skip the check. Run with a fixed Adam step, this kernel raised the marginal log-likelihood overall ($-18978 \rightarrow -16652$ nats on a 400-chain fit) while violating monotonicity by 547 nats at $\eta = 0.02$ and 1452 at $\eta = 0.05$ — at which point EM’s one guarantee is gone and a diverging run looks exactly like a slow one. Backtracking the step until Q actually increases restores exact monotonicity (0 violation at both rates) at negligible cost, since $Q = \langle \Xi, \log K \rangle$ is one contraction of a matrix already in hand. It is also diagnostic: at $\eta = 0.05$ the backtracked run stalls at -17392 instead of silently descending, which is the learning rate reporting itself as too large.

Remark 8 (More expressive is not better here). On the Laplace chain with 400 observed noisy chains ($M = 201$, 250 EM iterations), the neural kernel with 873 parameters is dominated on *both* axes by the 13-parameter mixture kernel of Rung 3: training log-likelihood -16781 against -16656 , and held-out relative score error 0.052 against 0.026. It has not bought expressivity that this data needs; it has bought optimization difficulty, since a gradient M-step under a monotonicity safeguard makes slower progress per iteration than a closed-form one.

The one place the network does edge ahead is instructive and not a recommendation: run *without* the safeguard of Remark 7 at $\eta = 0.05$, it attains -16652 , marginally above the mixture’s likelihood, while its held-out error stays worse at 0.059 — ordinary overfitting, reached by a procedure that violated monotonicity by 1452 nats to get there.

Rung 4 is the right tool when the transition genuinely has state-dependent shape that Rung 3 cannot express. On data where a linear autoregression with a free innovation law is correct, it costs two orders of magnitude in parameters and returns nothing. The efficiency argument of Section 5 is an argument for matching the hypothesis class to the structure — and it applies to the structured method’s own internals, not only to the baseline it is being compared against.

5 Why this should beat a score network

5.1 The decoupling

The structural claim is one sentence: *the learned parameters live on the clean chain, and the noise level lives in the likelihood*. The object being estimated, K_θ , is a function on $\mathbb{R} \times \mathbb{R}$ with no dependence on t . Once $\hat{\theta}$ is fixed, the denoiser at any noise level is obtained by running BP with the likelihood factors (7) at that level — exactly, and with no further fitting. The estimation problem and the diffusion schedule are decoupled.

A denoising-score-matching network has no such decoupling. It represents $(x, t) \mapsto m$ as a function on $\mathbb{R}^n \times \mathbb{R}_+$; it must be trained across the whole schedule at once; and the target it is fitting changes with t . Nothing in the architecture knows that the chain is Markov, so the conditional independencies must be recovered from data.

5.2 Risk decomposition

Write $m_\theta(x, t)$ for the exact posterior mean under prior p_θ . For the BP estimator the excess risk at level t decomposes as

$$\underbrace{\|m_{\hat{\theta}}(\cdot, t) - m_{\theta^*}(\cdot, t)\|}_{\text{total}} \leq \underbrace{\|m_{\hat{\theta}} - m_{\theta^\dagger}\|}_{\text{estimation}} + \underbrace{\|m_{\theta^\dagger} - m_{\theta^*}\|}_{\text{family misspecification}}, \quad (27)$$

where $\theta^\dagger$ is the population optimum within the chosen family. Crucially there is *no third term*: given a prior, BP returns the exact Bayes denoiser, so there is no approximation error from a function class. The misspecification term is zero when the family contains the truth (Rungs 1–2 on matching data) and is the observed error floor otherwise (Rung 3 on Laplace data, where it decreases with C). The estimation term is parametric.

Proposition 5 (Sensitivity of the BP denoiser; a delta method). *Suppose $\theta \mapsto \log K_\theta$ is C^1 and that, for t in a compact interval $[t_{\min}, t_{\max}]$ with $t_{\min} > 0$, the posterior variances are bounded. Then $m_\theta(x, t)$ is differentiable in θ with*

$$\frac{\partial m_{\theta,i}(x, t)}{\partial \theta_p} = \text{Cov}_{p_\theta(\cdot|x)} \left(a_i, \sum_{l=2}^n \frac{\partial \log K_\theta(a_l | a_{l-1})}{\partial \theta_p} \right), \quad (28)$$

a posterior covariance computable from the same beliefs BP already produces. Consequently, if $\hat{\theta} - \theta^\dagger = O_P(N^{-1/2})$, then $\sup_{t \in [t_{\min}, t_{\max}]} \|m_{\hat{\theta}}(\cdot, t) - m_{\theta^\dagger}(\cdot, t)\| = O_P(N^{-1/2})$.

Proof sketch. Differentiate $m_{\theta,i} = \int a_i p_\theta(a | x) da$ under the integral. Since $\partial_{\theta_p} p_\theta(a | x) = p_\theta(a | x) [\partial_{\theta_p} \log p_\theta(a) - \mathbb{E}_\theta \partial_{\theta_p} \log p_\theta(a)]$ and $\log p_\theta(a)$ is the sum of log-transitions, (28) follows. The uniformity statement is the mean value theorem applied along the segment from $\theta^\dagger$ to $\hat{\theta}$, using that the covariance (28) is bounded on the compact t -interval. $\square$

The content of Proposition 5 is the word *uniformly*: one $N^{-1/2}$ -consistent parameter estimate delivers an $N^{-1/2}$ -accurate denoiser at *every* noise level simultaneously. The network gets no such statement; its error must be paid for separately in every region of (x, t) it is asked about.

It is worth recording that the obvious corollary of this — that the network should visibly break at noise levels outside its training schedule — is *false*, and experiment 07 Part 3 was designed expecting it. Across thirteen levels from $t = 0.02$ to $t = 3.2$ the network shows no cliff at the schedule boundary, and the EM-BP advantage (a factor of 4.5 to 19) does not separate in-schedule

from out-of-schedule levels at all. Diffusion time enters through smooth features and the target moves smoothly with t , so a network handles that one direction adequately: the estimation difficulty lives in x -space, not in t -space. What survives is the weaker but real statement that EM-BP has no schedule to leave — its error varies smoothly across the whole range because its parameters are properties of the clean chain and BP supplies each t exactly.

5.3 What the comparison must control

An efficiency claim is only as good as the baseline, so the experiments are set up to favour the network wherever there was a choice:

1. **Data budget** is the number of *clean chains* N . The network trains on paired (a, x) with a fresh noise draw at every gradient step, so it may consume unlimited noise realizations of those N chains. EM sees one noisy realization of each chain, at one level, and never sees a clean chain. The two methods are therefore *not* on equal information: EM is given strictly less.
2. **Objective** is denoising score matching, whose minimizer is exactly $\mathbb{E}[a \mid x, t]$ — the same target BP computes. The network is not handicapped by a surrogate loss. Both standard parameterizations are trained and reported: noise prediction ($\|z_\phi - z\|^2$, the usual diffusion recipe) and clean-signal prediction ($\|a_\phi - a\|^2$). They share a minimizer but not a finite-sample loss, the two differing by the weight α_t^2/Δ_t ; noise prediction additionally recovers the mean as $(x - \sqrt{\Delta_t} z_\phi)/\alpha_t$, whose $\sqrt{\Delta_t}$ prefactor suppresses network error exactly where signal prediction is weakest. Resting the comparison on whichever parameterization happens to lose would not be evidence of anything.
3. **Capacity and training budget** are swept, so the comparison cannot hinge on an undersized or undertrained network.
4. **Evaluation** is on held-out chains against exact grid BP under the true prior, with the score/mean identity $\hat{s} - s = (\alpha_t/\Delta_t)(\hat{m} - m)$ verified to machine precision on every row as an implementation guard.

6 Limitations

1. **Inference cost is the honest loss.** Grid BP costs $O(nM^2)$ per chain per evaluation; a network forward pass is a few matmuls. Since reverse diffusion evaluates the denoiser at every integration step, this is not a detail. The measured slowdown is $211\times$ to $320\times$ at $M = 401$ (experiment 07 Part 4, 1.2–2.0 ms per chain against 4–10 μ s for a forward pass) and is reported as prominently as the wins. The natural responses — distilling the fitted BP denoiser into a network, or using the Gaussian-projected BP of Layer 2 at inference — are the obvious next step, not something claimed here.
2. **Everything rests on the chain assumption.** If the prior is not Markov, the estimator is misspecified in a way no amount of data fixes. Layer 4 showed the correction is exactly rank one for AR(1)-plus-global-latent priors, which suggests a hybrid, but that is not tested here.
3. **Seeding.** While preparing this layer we found that the package’s seeding helper derived its seeds from Python’s builtin `hash`, which is salted per process for strings (PEP 456). Since every call mixes in a string tag, no experiment in the package was bit-reproducible from a fresh interpreter, contrary to what its documentation claimed. The paired “common random numbers” property

that the method comparisons actually rely on held *within* each run, so previously reported numbers stand as measurements; but they were not reproducible. The helper now uses a fixed digest and the experiments here were re-run under it. Earlier layers’ committed outputs have not been regenerated.

4. **The state space must be represented.** The M^2 in every cost is the price of continuous messages, and it is what makes the extension beyond scalar-valued sites nontrivial. Nothing in Section 2 requires a grid — Ξ is defined by (15) for any message representation — but a grid is what is implemented.
5. **One dataset, one structure.** The experiments use a single chain family (linear AR(1) with non-Gaussian innovations) at $n = 32$. The claim “EM beats a score network” is demonstrated in this setting, not in general; in particular the network’s disadvantage would shrink if the prior were complicated enough that the chain family were itself a poor description.
6. **The baseline is a vanilla network, as specified — and that is a real restriction.** The comparison uses a fully connected MLP, sweeping width and training budget and reporting both standard parameterizations (noise prediction and clean-signal prediction). It does *not* use an architecture that encodes sequence structure. A temporal convolution or a U-Net would carry a locality prior of its own and should be expected to close part of the gap; what it would not acquire is the exactness of (27) or the uniformity in t of Proposition 5. Establishing where a structured *architecture* lands relative to a structured *estimator* is the natural next comparison and is not settled here.

7 Relation to the rest of the project

Layer 2 established that BP computes the exact score for a *known* Markov prior, and that Gaussianity buys closure of the message representation, not exactness. Layer 4 established that for approximately Markov priors the score correction is low rank, and that a residual network on top of a frozen Markov-BP score is far more efficient than a direct score network. This layer completes the sequence by removing the last thing that was handed to BP — the prior itself — and shows that the same tree structure which made the score exact also makes the *learning* exact, in the sense that the E-step involves no approximation.

Read together the three results form one argument: structural knowledge about the data should enter the algorithm, not the training set. Layer 2 puts it in the inference, Layer 4 puts the residual in a network, and Layer 5 puts the estimation in a low-dimensional parameter. Each step converts a piece of what a black-box denoiser would have to learn into something that is computed instead.

A Notation

symbol	meaning
$a \in \mathbb{R}^n$	clean sequence (latent)
$x \in \mathbb{R}^n$	noisy observation at level t
α_t, Δ_t	$e^{-t}, 1 - e^{-2t}$
$K_\theta(a' a)$	transition kernel of the clean chain
$\ell_i^t(a_i; x_i)$	per-site likelihood $\mathcal{N}(x_i; \alpha_t a_i, \Delta_t)$
L_i, R_i	forward / backward BP messages (excluding the local factor)
$b_i, b_{i-1,i}$	single-site and pairwise posterior beliefs
$u_1..u_M, w$	message grid and quadrature weights
$\Xi \in \mathbb{R}^{M \times M}$	expected pair-count matrix (15)
$W = \sum \Xi$	total edge count $N(n - 1)$
$Q(\theta \theta')$	expected complete-data log-likelihood
J_t	Fisher information of θ per noisy chain
$m(x, t), s(x, t)$	posterior mean (denoiser) and score

B Implementation map

object in this note	code
E-step, Ξ , evidence	<code>src/em.py:e_step, e_step_multi</code>
$t \rightarrow 0$ limit (clean data)	<code>src/em.py:clean_statistics, fit_clean</code>
EM driver, monotonicity trace	<code>src/em.py:fit_em, EMTrace</code>
Fisher identity (17)	<code>src/em.py:q_gradient</code>
Rungs 1–4 and their M-steps	<code>src/kernels.py</code>
BP denoiser / DSM baseline	<code>src/denoiser.py</code>
Correctness, information, rates	<code>experiments/exp_06_em_parameter_recovery.py</code>
EM-BP vs score network	<code>experiments/exp_07_em_vs_score_network.py</code>